

Plan1

Cohort 3 Group 8

Henry Gillett, Rishi Thiahulan, Isaac Mileham, Ida Kenyon,
Lenny Schwarz, Viktor Barabash, Andrii Kuznietsov

Justification of development & collaboration tools & methods:

Git

Our choice of Git as the version control system because of its distributed nature, which complements the team's Agile workflow. Git enables collaborative integration but also iterative updates, which allows the team to work independently around their timetables. Efficient branching and merging capabilities were key considerations as they allow for parallel development and experimentation, which can then be easily merged into the main project. This was essential as experimentation was a critical aspect of our development process. Further, integration with our teams' tooling, GitHub and IntelliJ IDE, helped streamline our process of propagating code. In contrast, the other main version control solution Subversion employs a centralised model that introduces a single point of failure, the server. Given the limited time frame we had, it would be too much of a risk. Git's distributed approach alleviated this via multiple redundant copies of the repository.

Scrum

We chose to follow the Scrum framework to structure our Agile process. Work was organised into weekly sprints, we held twice-weekly stand-ups for progress tracking, and retrospectives for continued improvement. The iterative nature of Scrum and defined roles helped promote accountability and adaptability within our team; this has helped us respond efficiently to the changing requirements of the client and project. Alternative frameworks such as Kanban while offering more visual workflow managements, lack in defined roles and clear milestone tracking which was crucial for our team in controlling pace as we worked towards assessment 1. In light of these points, and given the project's dynamic nature, Scrum was the most suitable choice.

Asana

Asana was selected as our project management tool because it provided robust task tracking and team collaboration features that aligned directly with our Scrum framework. Its timeline and dependency tracking aided sprint planning and progress visualisation, allowing us to monitor our tasks through each stage of the sprint cycle. Compared to alternative choices like Trello, Asana offered stronger reporting and added automation, which allowed us to maintain a constant pace of Scrum iteration, ensuring that our sprint objectives are clearly defined, tracked, and met.

Discord

Our team used Discord as the primary communication platform. It was chosen for its widespread adoption within the team already (and thus familiarity), and its flexibility compared to alternatives such as Slack. Whilst Slack does support text-based channels, Discord extends this with the addition of voice channels, video calls and screen-sharing features. These enable real-time remote debugging, code reviews and remote meetings (e.g. during consolidation week). This supported our Agile workflow's emphasis on collaboration and rapid feedback. Discord's support for 'GitHub Webhooks' permits us to

create a commit feed, ensuring all members of the implementation team are aware of developments as soon as they are made, improving coordination and reducing communication delay. We also broke down each team into separate channels, which helped keep the communications clear within teams; we also had a *general* channel to allow for inter-team communication.

Google Drive

Google Drive was adopted for its strengths in cloud-based storage and integration with Google Docs, supporting the team's need for synced and accessible documentation. Its real-time collaboration allows for parallel contribution to project documentation, aligning with Agile's collaborative principles. Version history provides the team with a safeguard against accidental overwrites, and improves accountability within tracking document changes. We preferred Google Drive over alternatives like Microsoft OneDrive for its more intuitive web interface, smoother editing experience, and integration with the university's Google Workspace.

2D Design

Two-dimensional graphics were chosen for their mutual benefits to both developers and players. They are suitable for developers new to gamemaking as when compared to three dimensions, a reduced field to plan, design, and code for makes development easier. For video-game beginners, two dimensions are conceptually easier to interact with, as they require fewer physical input mappings to negotiate when moving around an environment. A reduced dimension is also less resource intensive to build and run, improving developer experience with shorter build times and more responsive playtests; further aligning our game with the client's expectations for it to be cross-platform and run without specialist hardware, as a 2D game should run across all platforms without performance issues.

Game Engine

We chose libGDX as our game engine due to its rich documentation, community support, and extensibility when producing 2D games. The community maintains tutorials and accessible forums for beginners to ask questions and receive feedback, supporting our implementation team who have never used the engine before. It is cross-platform and has low overhead, allowing us to satisfy our requirement for the game to work on a variety of devices, including low-powered systems. An alternative approach would have been to use jMonkeyEngine or to interface with the Lightweight Java Game Library directly, instead of through the structured layer which is libGDX. However, these choices would have been illogical for us as the complex development challenges these alternatives bring would have been unmanageable and unnecessary within the scope of this project.

Team Organisation methods:

The first part of our working structure that we implemented were weekly team meetings with the whole group to cover updates to the project and allow for communication within the whole team. This is because it allows the whole team to get a view of what is happening within the project and for us to be able to update plans with every team member's input. It also suited the group very well as it allowed for reassurance and clarity on every part of the project getting completed.

The next large working structure that we implemented was splitting into smaller groups and splitting the deliverables between those groups. This meant that every work package and deliverable had its own group with at least one person and one backup team member for the smallest work packages, for example the website, and all team members on the larger or more collaborative work packages, for example testing. This is because it allows the deliverables to be completed with less overheads for assigning individual tasks to people and it reduces any potential confusion with what people are working on. In addition, we allowed groups to work autonomously, with exception for the weekly meetings with the whole group, so they could meet if necessary or just work online depending on the task and it wouldn't effect or be reliant on the whole teams availability allowing for individual tasks to be completed in a more agile and effective way.

Key tasks:

Task ID	Name	Description	Start	End	Dependency
T1.1	PDF docs for Website	Linking deliverables in website	30/10	10/11	WP2, WP3, WP4, WP5
T1.2	Structuring webpages	Making website clear	29/9	17/10	
T2.1	Software tools	Deciding tools	29/9	9/10	
T2.2	Collaboration software tools	Deciding collaboration tools	29/9	9/10	
T2.3	Group methodology	Deciding methodology	16/11	23/10	
T3.1	Risk identification	Identifying risks	23/11	30/10	T.4.2.1
T3.2	Risk ownership	Assigning risks for overview	31/10	6/11	T3.1
T4.2.1	System requirements	Outline requirements of systems	9/10	16/10	M4.1
T4.2.2	Requirements reevaluation	Revisiting requirements	23/10	30/10	T4.2.1
T5.1	Updating versions	Revisiting architecture	23/10	30/10	WP5.1
T5.2	Link to requirements	Designing based on requirements gathered	30/10	10/11	WP4, WP5.1
T5.1.1	Structure diagrams	Outlining structure of project	13/10	30/10	WP4, T5.1.2
T5.1.2	Tool justification	Justify architecture tools	30/10	10/11	
T6.1	Learn software	Learn software tools	9/10	16/10	T2.1
T6.2	Unit tests	Write unit tests	7/11	10/11	WP6.1
T6.1.1	Coding functionality	Program functionality	16/10	6/11	WP5
T6.1.2	Graphical design	Create visual interface	16/10	6/11	WP5

Evolution of plan

The plan has changed throughout the project with the intention of making it easier to track the progress being made to different tasks. A key example of this is in the week 3 gantt chart where both the Architecture and Implementation events were split into smaller, weekly segments to allow more in depth tracking of their completion. Other changes made were often in response to the progress made during a week. For example, in week 4 the method planning finalisation task was added to allow the method planning team to revisit and re-evaluate the method planning at a later date. Changes were also made as we went along the project and new tasks became obvious. For example, during consolidation week we added the Design presentation task as it became clear that we would need extra time to start the presentation before the 10th November deadline.

Blue - completed task
Grey - uncompleted task
Red - current week

